

VoiceCart AI — Improvement Plan

Action plan to close accuracy gaps, add proof-of-work, and strengthen the submission before final delivery. Prepared for implementation via Antigravity.

Repo	github.com/MAYANK479/voice_cart_ai
Live URL	voicecommandai.netlify.app
Stack	React 18 + TypeScript + Vite, Web Speech API, Vitest, GitHub Actions
Goal	Fix factual/technical inaccuracies, add real proof of functionality, reduce "AI-polish" red flags, tighten docs

CRITICAL

Fix before resubmitting — factual/technical error or missing proof

HIGH

High leverage, do next

1. Critical Fixes (do these first)

These are factual or technical inaccuracies in the current README that a technical reviewer will catch immediately. They must be corrected regardless of anything else on this list.

1.1. Correct the "100% offline" claim

CRITICAL

Files/area: README.md (top summary + write-up section)

Why: webkitSpeechRecognition in Chrome/Edge streams audio to Google's servers for transcription — it is not offline. Stating this incorrectly is worse than not mentioning it, since it signals either a misunderstanding of the API or an oversold README.

What to do:

- Remove or rewrite the "100% offline" line in the README executive summary.
- Replace with an accurate statement, e.g. "runs entirely client-side with no custom backend server; speech recognition uses the browser's built-in Web Speech API, which requires an internet connection in Chromium browsers."
- Add a short 'Known Limitations' note (see 3.2) instead of an offline claim.

1.2. Reframe "NLP" as rule-based parsing, not ML

CRITICAL

Files/area: README.md, src/services/nlpParser.ts (doc comments)

Why: The current parser is a deterministic keyword/regex matcher, not a trained NLP/ML model. The README's language implies more than is actually implemented. Any technical interviewer will ask "what model are you using?" — the honest answer should already be reflected in the docs.

What to do:

- Rename references from "NLP pipeline" to "rule-based intent & entity parser" where accurate, or clearly label it as "deterministic NLP (pattern/keyword-based, no ML model)".
- Add a short paragraph explaining the tradeoff: deterministic parsing was chosen for zero-latency, zero-cost, offline-tolerant behavior vs. calling an external LLM.
- Optional stretch (see 2.3): wire in a real free-tier LLM call as a fallback for phrases the rule parser can't confidently handle, which then genuinely justifies "AI-powered" in the title.

1.3. Remove or shrink the "Rubric Compliance" table

CRITICAL

Files/area: README . md

Why: A table that maps your own features directly onto the evaluator's stated grading criteria reads as gaming the scorecard rather than confidence in the work, and can make the whole README feel written for the algorithm rather than for a human reviewer.

What to do:

- Delete the "Technical Assessment Rubric Compliance" table entirely.
- Let the feature list, architecture diagram, and demo clip (section 2.1) carry that weight instead.

2. High-Value Additions

These add real proof of functionality and depth, and are the highest-leverage use of remaining time — recruiters spend under two minutes per submission and often will not grant mic permissions themselves.

2.1. Add a real demo clip to the top of the README

HIGH

Files/area: README.md, docs/screenshots/ (replace SVG banner)

Why: The current "showcase" images are generated SVG banners, not actual screenshots or footage of the running app. A 30-60 second screen recording is the single highest-ROI addition — it proves the voice flow actually works without the reviewer needing to test it themselves.

What to do:

- Record a short screen capture (Loom, OBS, or the OS screen recorder) walking through: add an item by voice → restock suggestion → substitute suggestion → price-filtered search → remove an item.
- Convert to an optimized GIF (or embed the video link) at the very top of the README, above the badges.
- Replace the static SVG "dashboard_banner" and "features_grid" with 2-3 real PNG screenshots of the actual UI in each major state (list view, NLP Lab, Insights).

2.2. Add a 'Known Limitations' section

HIGH

Files/area: README.md

Why: Proactively naming limitations reads as senior-level self-awareness rather than weakness, and pre-empts the obvious follow-up questions a reviewer would otherwise ask.

What to do:

- State that Web Speech API voice recognition is Chrome/Edge only (no Firefox/Safari support).
- State that voice recognition requires an internet connection despite the app's UI running client-side.
- Note that the product catalog / price data is mock/seed data, not a live pricing feed, and where it was sourced from.
- Note any known parsing edge cases the rule-based parser doesn't yet handle well.

2.3. (Stretch) Wire in a real free-tier AI/LLM call

HIGH

Files/area: src/services/nlpParser.ts, new src/services/llmFallback.ts

Why: The assignment explicitly allows use of any free-tier AI/ML service. Adding a genuine model call — even just as a fallback when the rule-based parser has low confidence — upgrades the project from "keyword matching branded as AI" to something that actually uses AI, and directly answers the most likely technical follow-up question.

What to do:

- Pick a free-tier LLM endpoint (e.g. Groq, Gemini free tier, or OpenAI-compatible endpoint) callable client-side or via a lightweight serverless function.
- Trigger the LLM call only when the deterministic parser's confidence score falls below a threshold (keeps cost near zero and preserves the fast-path for common phrases).
- Document this fallback behavior clearly in the README and NLP Lab, including what happens if the API key/quota is unavailable (graceful degrade back to rule-based parsing, not a crash).

2.4. Verify and screenshot the CI badge status

HIGH

Files/area: .github/workflows/ci.yml, README.md

Why: The README links a CI badge; a reviewer who clicks through and finds a failing or stale run undermines the credibility of everything else in the README.

What to do:

- Confirm the latest GitHub Actions run is green on the `main` branch.
- Re-run `npm test` locally and paste the actual pass count into the README if it differs from the current claimed 25/25.

3. Polish & Submission Hygiene

Lower-effort items that reduce the risk of the submission being read as fully AI-generated with minimal hands-on understanding, and tighten it against the original brief.

3.1. Trim README tone / reduce badge & emoji density

MEDIUM

Files/area: README.md

Why: The current heavy badge/emoji/marketing tone, combined with only 4 total commits, can read as generated in one shot rather than iteratively built — inviting harder scrutiny in a follow-up interview.

What to do:

- Cut emoji section headers roughly in half; keep them only where they aid scanning.
- Shorten the executive write-up to strictly meet the brief's 200-word cap (verify actual word count).
- Keep the architecture diagram and voice-command table — these are genuinely useful, not filler.

3.2. Confirm full requirement coverage against the original brief

MEDIUM

Files/area: N/A – verification task

Why: The requirements PDF supplied to you appears to skip from section 4 straight to section 11, meaning sections 5-10 were not visible when this plan was written. Unverified coverage is a risk.

What to do:

- Get the complete, unedited version of the assessment brief from Unthinkable if possible.
- Cross-check every numbered requirement against a shipped feature; note any gaps explicitly in the README rather than leaving them silently unaddressed.

3.3. Add a short 'Data Sources' note

MEDIUM

Files/area: README.md, src/data/

Why: The brief allows collecting test data from public sources; stating where seed/catalog data came from shows the same diligence expected in the rest of the project.

What to do:

- Add one or two lines identifying whether product/price data is hand-authored mock data or pulled from a public dataset, and link the source if applicable.

3.4. Incremental commit hygiene (optional, if time allows)

MEDIUM

Files/area: git history

Why: Four commits total for a project this polished can look like a single AI-assisted dump. A few logically scoped follow-up commits for this plan's changes will naturally improve that signal without extra work.

What to do:

- Commit the fixes in this plan as separate, clearly-labeled commits (e.g. "fix: correct offline claim in README", "feat: add LLM fallback for low-confidence parses", "docs: add limitations section") rather than one giant commit.

Suggested order: Section 1 (all three items) → 2.1 and 2.2 → 2.4 → 2.3 if time remains → Section 3. Sections 1 and 2.1/2.2 alone meaningfully de-risk the submission even if nothing else gets done.